

class122 树上差分 笔记

树上问题专题五。前置：树上倍增与LCA（118）、链式前向星建图（059）、树形DP（078/079）、递归改迭代（118）、二分答案法（051）。

树上差分：点差分与边差分

树上差分解解决的核心问题：给很多「路径修改」操作（比如把 a 到 b 路径上的点权/边权都加 v），能不能用**单次操作 O(1)** 的代价完成，最后一遍 **DFS** 统一加工出所有点的正确值。

一切的前提是能快速求出路径两端点的**最低公共祖先 LCA**（倍增或 tarjan，见讲解118）。

点差分（改点权）

路径 $x \rightarrow y$ 上所有点权加 v，只需要做 **4 条操作**：

```
1 num[x] += v
2 num[y] += v
3 num[lca] -= v
4 num[father(lca)] -= v
```

最后一遍 DFS 汇总： $\text{num}[u] += \text{所有子节点的 num}$ 。

道理：x 加 v 会沿路径一路向上传递到 lca，y 加 v 也会传递到 lca，lca 处累计了两份 v，所以要减掉一份 v；但整棵子树还「盈余」了一个 v 往外冒，所以在 lca 的父亲处再减一个 v 把影响彻底抵消。根节点上方设一个虚拟 0 号节点（点权恒 0）承接这个减操作。

边差分（改边权）

路径 $x \rightarrow y$ 上所有边权加 v，只需要做 **3 条操作**：

```
1 num[x] += v
2 num[y] += v
3 num[lca] -= 2 * v
```

最后一遍 DFS 汇总： $\text{num}[u] += \text{所有子节点的 num}$ ；此时 $\text{num}[u]$ 的含义就是「u 到父节点这条边」被修改的增量（原始边权 + $\text{num}[u]$ 就是新边权）。

道理：点差分里 lca 处要减 v、lca 父亲处也要减 v，而边差分里 lca 到父节点这条边**本来就不该被修改**，所以把这两个减操作合并成 lca 处一次 $-2*v$ ，把影响在 lca 处就彻底消掉，不让它冒到 lca 的父边上去。

题目一：树上点差分模版（P3128）

题面

n 个节点形成一棵树，初始所有点权都是 0；给定 m 个操作 (a,b)，表示从 a 到 b 路径上所有点的点权 +1。所有操作完成后，返回树上的最大点权。

核心思路

裸的点差分模版，用倍增法求 LCA：

1. dfs1 建 ST 表 (deep 深度 + stjump 倍增跳表) ;
2. 每条操作做 4 条点差分 (两 endpoints +1, lca -1, lca 的父亲 -1) , lca 的父亲直接用 stjump[lca][0] 拿到;
3. dfs2 一遍 DFS 汇总点权, 最后取 num[1..n] 的最大值。

坑点

- java 递归版会爆栈 (C++ 能过) , 迭代版见「写法 2」, 逻辑完全一样。
- 根的父亲是虚拟 0 号点: stjump[根][0] = 0, num[0] 随便减不影响最终结果。
- 复杂度: 倍增建表 $O(n \log n) + O(m)$, tarjan 则是 $O(n) + O(m)$ 。

写法 1: 递归版

```
1  const int MAXN = 50001, LIMIT = 16;
2  int power, cnt;
3  int num[MAXN], head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
4  int deep[MAXN], stjump[MAXN][LIMIT];
5
6  int log2(int n) { int ans = 0; while ((1 << ans) <= (n >> 1)) ans++; return ans; }
7
8  void dfs1(int u, int f) {
9      deep[u] = deep[f] + 1;
10     stjump[u][0] = f;
11     for (int p = 1; p <= power; p++)
12         stjump[u][p] = stjump[stjump[u][p - 1]][p - 1];
13     for (int e = head[u]; e != 0; e = nxt[e])
14         if (to[e] != f) dfs1(to[e], u);
15 }
16
17 int lca(int a, int b) {
18     if (deep[a] < deep[b]) swap(a, b);
19     for (int p = power; p >= 0; p--)
20         if (deep[stjump[a][p]] >= deep[b]) a = stjump[a][p];
21     if (a == b) return a;
22     for (int p = power; p >= 0; p--)
23         if (stjump[a][p] != stjump[b][p]) { a = stjump[a][p]; b = stjump[b][p]; }
24     return stjump[a][0];
25 }
26
27 void dfs2(int u, int f) {
28     for (int e = head[u], v; e != 0; e = nxt[e]) { v = to[e]; if (v != f) dfs2(v, u); }
29     for (int e = head[u], v; e != 0; e = nxt[e]) { v = to[e]; if (v != f) num[u] +=
num[v]; }
30 }
31 // 每条操作: num[u]++, num[v]++, num[lca(u,v)]--, num[stjump[lca][0]]--
```

写法 2：迭代版

dfs1、dfs2 递归改迭代（栈里存 `(u, f, e)` 三元组，`e == -1` 表示第一次来到 `u`），其余完全一样。改法见讲解118。

题目二：松鼠的新家 (P3258)

题面

n 个节点形成一棵树，给定一个点编号数组 `travel` 表示松鼠依次要去往的地点。松鼠每走到一个节点必须拿一个糖果，否则停止前进；到达最后一个地点时不需要吃糖果。打印每个节点上至少准备多少糖果才能让松鼠走完。

核心思路

点差分 + tarjan 批量求 LCA：

1. 相邻地点 `(travel[i], travel[i+1])` 看成一条路径，全部做点差分（两端点 $+1$ ，lca -1 ，lca 父亲 -1 ）；
2. tarjan 一次性求出所有相邻点对的 LCA（存进 `ans`），并用 `father` 数组单独记录每个节点的真实父亲（不是并查集的父亲）；
3. 一遍 DFS 汇总点权后，除了第一个地点，其余 `travel[2..n]` 的糖果数都 -1 ：中间节点是因为「作为上一条路的终点、又作为下一条路的起点」被重复算了一次；最后一个节点是因为题目要求不需要糖果。

坑点

- 这里 LCA 用 tarjan（讲解118），注意 `father` 数组记录的是真实树上的父节点，`unionfind` 才是 tarjan 的并查集，两者要分开。
- 最后 -1 从 `travel[2]` 到 `travel[n]`，第一个地点 `travel[1]` 不减。
- java 递归版会爆栈，需改迭代版；tarjan、find、dfs 都要改（find 的路径压缩也用栈模拟）。

```
1  const int MAXN = 300001;
2  int travel[MAXN], num[MAXN];
3  int headEdge[MAXN], edgeNext[MAXN << 1], edgeTo[MAXN << 1], tcnt;
4  int headQuery[MAXN], queryNext[MAXN << 1], queryTo[MAXN << 1], queryIndex[MAXN << 1], qcnt;
5  bool visited[MAXN];
6  int unionfind[MAXN], father[MAXN], ans[MAXN];
7
8  int find(int i) { return i != unionfind[i] ? unionfind[i] = find(unionfind[i]) : unionfind[i]; }
9
10 void tarjan(int u, int f) {
11     visited[u] = true;
12     for (int e = headEdge[u], v; e != 0; e = edgeNext[e]) { v = edgeTo[e]; if (v != f) tarjan(v, u); }
13     for (int e = headQuery[u], v; e != 0; e = queryNext[e]) {
14         v = queryTo[e];
15         if (visited[v]) ans[queryIndex[e]] = find(v);
16     }
17     unionfind[u] = f;
18     father[u] = f;
19 }
20
```

```

21 void dfs(int u, int f) {
22     for (int e = headEdge[u], v; e != 0; e = edgeNext[e]) { v = edgeTo[e]; if (v != f)
    dfs(v, u); }
23     for (int e = headEdge[u], v; e != 0; e = edgeNext[e]) { v = edgeTo[e]; if (v != f)
    num[u] += num[v]; }
24 }
25
26 void compute(int n) {
27     tarjan(1, 0);
28     for (int i = 1; i < n; i++) {
29         int u = travel[i], v = travel[i + 1], lca = ans[i], lcafather = father[lca];
30         num[u]++; num[v]++; num[lca]--; num[lcafather]--;
31     }
32     dfs(1, 0);
33     for (int i = 2; i <= n; i++) num[travel[i]]--;
34 }

```

题目三：最小化旅行的价格总和（LeetCode 2646）

题面

n 个节点形成一棵树，每个节点有点权，给定很多路径（每条路径有开始点、结束点，代价 = 沿途点权和，所有路径代价之和就是旅行总价）。你可以选择把**某些点**的点权减半，但**选择的点不能相邻**。返回旅行价格总和最少能是多少。

核心思路

点差分（统计次数） + 树形 DP（讲解078/079）：

1. 用一个**次数数组** `num` 做点差分，统计每个点被多少条路径经过（注意：差分的是「经过次数」，不是原有点权 `price`）。
2. 树形 DP 两状态：`no` = u 一定不减半时以 u 为头的子树最小花费，`yes` = u 一定减半时以 u 为头的子树最小花费：

```

1 | n = price[u] * num[u];          // u 不减半，自己贡献全价 * 经过次数
2 | y = (price[u] / 2) * num[u];    // u 减半，自己贡献半价 * 经过次数
3 | 对每个孩子 v: n += min(no_v, yes_v); y += no_v;    // 减半则孩子都不能减半

```

3. 最终答案 `min(no_根, yes_根)`。

坑点

- LeetCode 编号从 0 开始，代码里统一 +1 调整成从 1 开始。
- 差分的是 `num`（次数）数组，不是 `price`（原有点权）数组，别改错。
- LCA 有两种写法：倍增（dfs1 建表）和 tarjan，功能一样。

```

1 | const int MAXN = 51, LIMIT = 6;
2 | int power, cnt;
3 | int price[MAXN], num[MAXN], head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
4 | int deep[MAXN], stjump[MAXN][LIMIT];
5 | int no, yes;
6 |

```

```

7 void dp(int u, int f) {
8     int n = price[u] * num[u];
9     int y = (price[u] / 2) * num[u];
10    for (int e = head[u], v; e != 0; e = nxt[e]) {
11        v = to[e];
12        if (v != f) {
13            dp(v, u);
14            n += min(no, yes);
15            y += no;
16        }
17    }
18    no = n;
19    yes = y;
20 }
21 // 先点差分求出 num[1..n] (每条路径: num[u]++, num[v]++, num[lca]--, num[stjump[lca][0]]--
22 // 再 dp(1, 0), 答案 min(no, yes)

```

题目四：使图不连通的方法数（POJ 3417，边差分实战）

题面

n 个节点、 $n-1$ 条「老边」连成一棵树，再额外加 m 条「新边」。你可以切断两条边让图不连通，且**必须切一条老边 + 一条新边**。返回方法数。

核心思路

边差分 + 结构分析。把每条新边 (u,v) 看成一次边差分（ $\text{num}[u]++$, $\text{num}[v]++$, $\text{num}[\text{lca}]-=2$ ），DFS 汇总后，每条老边的「边权」= 它被几条新边形成的环覆盖（ $w = \text{num}[\text{子节点}]$ ）：

- $w == 0$ ：这条老边不在任何新环上，切断它图就断开，再任选一条新边都行 → 贡献 m 种；
- $w == 1$ ：这条老边被唯一一条新边覆盖，切断它后必须再切「那条新边」才能断 → 贡献 1 种；
- $w > 1$ ：被多条新边覆盖，只切一条新边不够断 → 贡献 0 种。

所有老边贡献相加就是答案。

坑点

- 老边原始边权都是 0 （ $w = 0 + \text{num}[v]$ ），边权 = 点权 $\text{num}[v]$ 汇总值。
- 本题递归不改迭代也能过（POJ 数据不大）。
- 用倍增求 LCA，复杂度 $O(n \log n + m + n)$ 。

```

1  const int MAXN = 100001, LIMIT = 17;
2  int n, m, power, cnt, ans;
3  int num[MAXN], head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
4  int deep[MAXN], stjump[MAXN][LIMIT];
5
6  void dfs2(int u, int f) {
7      for (int e = head[u], v; e != 0; e = nxt[e]) { v = to[e]; if (v != f) dfs2(v, u); }
8      for (int e = head[u], v, w; e != 0; e = nxt[e]) {
9          v = to[e];
10         if (v != f) {
11             w = 0 + num[v];

```

```

12         if (w == 0) ans += m;
13         else if (w == 1) ans += 1;
14         num[u] += num[v];
15     }
16 }
17 }
18 // 每条新边: num[u]++, num[v]++, num[lca(u,v)] -= 2

```

题目五：运输计划（P2680）

题面

n 个节点、 $n-1$ 条边（每条边有正数边权）连成一棵树，给定很多运输计划 (a,b) （代价 = 沿途边权和，计划之间互不干扰）。你只能选一条边把边权变成 0，目标是让所有运输计划代价的**最大值尽量小**，返回这个最小化的最大值。

核心思路

二分答案法（讲解051） + 边差分：

- 先用 tarjan 一遍求出每个计划的两端点 LCA、代价 `cost[i]` (`dist[u]+dist[v]-2*dist[lca]`)，以及 `maxCost`（最大代价）。
- 答案落在 `[0, maxCost]`，二分 `limit`，用 `f(limit)` 判断「能否让所有计划代价 $\leq$ limit」：
 - `atLeast = maxCost - limit`：某条边至少要能减掉这么多边权；
 - 对所有 `cost[i] > limit` 的**超标**计划做边差分（`num[u]++`, `num[v]++`, `num[lca]-=2`），`beyond` = 超标计划数；
 - 若 `beyond == 0` 直接 true；否则找一条边同时满足：**被所有超标计划经过**（`num[该边]==beyond`）且**边权 $\geq$ atLeast**。存在则 true。
- 二分收窄得到最小可行 `limit`。

坑点

- 边差分「边被经过的次数」体现在子节点上：`num[u]` 汇总后代表 u 到父节点这条边被超标计划经过的次数。
- 需要两个数组：`dist`（根到每个点的距离，tarjan 过程中顺手更新）、`father`（真实父节点）。`distance` 数组名会和 `std::distance` 冲突，C++ 里改名 `dist`。
- 递归版（tarjan + dfs）在 java 会爆栈，需改迭代版；逻辑完全一样。
- 复杂度：tarjan $O(n+m)$ ，二分 $O(\log \text{maxCost})$ 次，每次 $O(n)$ 。

```

1  const int MAXN = 300001, MAXM = 300001;
2  int n, m, maxCost, atLeast, beyond;
3  int num[MAXN];
4  int headEdge[MAXN], edgeNext[MAXN << 1], edgeTo[MAXN << 1], edgeweight[MAXN << 1],
   tcnt;
5  int headQuery[MAXN], queryNext[MAXM << 1], queryTo[MAXM << 1], queryIndex[MAXM << 1],
   qcnt;
6  bool visited[MAXN];
7  int unionfind[MAXN], quesu[MAXM], quesv[MAXM];
8  int dist[MAXN], lca[MAXM], cost[MAXM];
9
10 void tarjan(int u, int f, int w) {

```

```

11     visited[u] = true;
12     dist[u] = dist[f] + w;
13     for (int e = headEdge[u], v; e != 0; e = edgeNext[e]) { v = edgeTo[e]; if (v != f)
tarjan(v, u, edgeweight[e]); }
14     for (int e = headQuery[u], v, i; e != 0; e = queryNext[e]) {
15         v = queryTo[e];
16         if (visited[v]) {
17             i = queryIndex[e];
18             lca[i] = find(v);
19             cost[i] = dist[u] + dist[v] - 2 * dist[lca[i]];
20             maxCost = max(maxCost, cost[i]);
21         }
22     }
23     unionfind[u] = f;
24 }
25
26 bool dfs(int u, int f, int w) {
27     for (int e = headEdge[u], v; e != 0; e = edgeNext[e]) {
28         v = edgeTo[e];
29         if (v != f) if (dfs(v, u, edgeweight[e])) return true;
30     }
31     for (int e = headEdge[u], v; e != 0; e = edgeNext[e]) { v = edgeTo[e]; if (v != f)
num[u] += num[v]; }
32     return num[u] == beyond && w >= atLeast;
33 }
34
35 bool f(int limit) {
36     atLeast = maxCost - limit;
37     memset(num + 1, 0, n * sizeof(int));
38     beyond = 0;
39     for (int i = 1; i <= m; i++)
40         if (cost[i] > limit) { num[quesu[i]]++; num[quesv[i]]++; num[lca[i]] -= 2;
beyond++; }
41     return beyond == 0 || dfs(1, 0, 0);
42 }
43 // 二分: l=0, r=maxCost; f(mid) 为真则记答案并 r=mid-1, 否则 l=mid+1

```